

## Java OOPs — Theory Notes

### 1. Object-Oriented Programming (OOP)

OOP is a programming paradigm based on **objects and classes**. It helps organize software by combining data and behavior into reusable units.

Java follows the object-oriented programming approach.

#### Main principles of OOP:

1. Encapsulation
  2. Inheritance
  3. Polymorphism
  4. Abstraction
- 

### 2. Class

A **class** is a blueprint or template used to create objects.

A class defines:

- Data/attributes
- Methods/behaviors
- Constructors
- Other members

A class itself does not represent a specific real-world entity; objects created from it do.

---

### 3. Object

An **object** is an instance of a class.

An object has:

- **State** — represented by data/variables
- **Behavior** — represented by methods
- **Identity** — distinguishes one object from another

Objects are created at runtime.

---

## 4. Encapsulation

Encapsulation is the process of **binding data and methods together into a single unit and controlling access to the data.**

In Java, encapsulation is commonly achieved using:

- Classes
- Private variables
- Public getter and setter methods
- Access modifiers

### Advantages:

- Data security
  - Controlled access
  - Data validation
  - Better maintainability
  - Reduced dependency between components
- 

## 5. Inheritance

Inheritance is a mechanism in which one class **acquires the properties and behavior of another class.**

The existing class is called:

- Parent class
- Superclass
- Base class

The new class is called:

- Child class
- Subclass
- Derived class

Inheritance promotes **code reusability.**

### Types supported through Java classes:

- Single inheritance

- Multilevel inheritance
- Hierarchical inheritance

Java does not support **multiple inheritance through classes** because it can create ambiguity.

Multiple inheritance can be achieved using interfaces.

---

## 6. Polymorphism

Polymorphism means **one entity having multiple forms**.

It allows the same operation or method to behave differently depending on the situation.

Java has two main types:

### Compile-time polymorphism

Achieved through **method overloading**.

### Runtime polymorphism

Achieved through **method overriding**.

---

## 7. Method Overloading

Method overloading means having **multiple methods with the same name but different parameter lists** within a class.

The difference can be based on:

- Number of parameters
- Type of parameters
- Order of parameters

Method overloading is resolved during **compile time**.

Changing only the return type does not constitute method overloading.

---

## 8. Method Overriding

Method overriding occurs when a subclass provides its **own implementation of a method already defined in its parent class**.

It is used to provide specialized behavior in the child class.

Method overriding is associated with:

- Inheritance
- Runtime polymorphism
- Dynamic method dispatch

The method signature must be compatible with the parent method.